SOFTWARE REQUIREMENTS SPECIFICATION

Algorithm Visualization System

SRS Document and UML Diagrams

Project Type	Educational software / web application
Prepared For	Academic mini-project / GitHub documentation
Version	1.0
Date	October 2026

Purpose: This document defines the requirements, scope, behavior, and UML design of a system that helps learners understand algorithms through step-by-step visual execution.

1. Introduction

1.1 Purpose

The purpose of the Algorithm Visualization System is to help students understand common algorithms by showing their execution step by step. The application displays the current state, pseudocode or source code, and the final result in a clear visual interface.

1.2 Scope

The system is intended for students and beginners learning data structures and algorithms. Initial algorithms may include Linear Search, Binary Search, Bubble Sort, Selection Sort, and Insertion Sort. Users can enter sample data, run or pause an animation, move through individual steps, and reset the demonstration.

1.3 Definitions

Algorithm: A finite sequence of steps used to solve a problem. **Visualization:** A visual representation of an algorithm's execution. **Step:** One state transition in the algorithm. **UI:** User Interface.

2. Overall Description

2.1 Product Perspective

The system is a standalone educational application. A browser-based implementation may use HTML, CSS, and JavaScript, with the algorithm engine separated from the user interface and visualization renderer.

2.2 User Classes

• Student / learner: selects an algorithm, supplies input, controls execution, and reviews results.
• Administrator / developer: maintains the algorithm catalogue and updates educational content.

2.3 Operating Environment

A modern web browser on a desktop, laptop, tablet, or mobile device. Internet access is required only if the application is hosted online; a local build may run offline.

2.4 Assumptions and Constraints

• Input must match the selected algorithm's requirements.
• Binary Search requires sorted input, or the application must clearly offer to sort it first.
• The first version focuses on learning and demonstration, not production-scale performance benchmarking.
• The interface should remain usable on common screen sizes.

3. Functional Requirements

ID	Requirement	Description
FR-01	Select Algorithm	The user shall be able to choose an algorithm from a list of supported algorithms.
FR-02	Enter Input	The user shall be able to enter an array, search key, or other algorithm-specific input.
FR-03	Validate Input	The system shall validate input and show a clear error message for invalid data.
FR-04	Start / Pause	The user shall be able to start and pause the visualization.
FR-05	Step Controls	The user shall be able to move to the next or previous step and reset the run.
FR-06	Visualize State	The system shall highlight active elements and update the display for each algorithm step.
FR-07	Show Code	The system shall display the relevant pseudocode or code and highlight the current operation.
FR-08	Show Result	The system shall display the final output and a brief explanation of the result.
FR-09	Complexity Details	The system should show time and space complexity where applicable.

4. Non-Functional Requirements

NFR-01 Usability: Controls and labels should be simple, consistent, and understandable to beginners.

NFR-02 Performance: For small educational inputs, a step update should normally appear without noticeable delay.

NFR-03 Reliability: The system should reset state correctly and avoid displaying stale results from a previous run.

NFR-04 Compatibility: The interface should support current major browsers and responsive layouts.

NFR-05 Accessibility: Controls should have readable labels, adequate contrast, and keyboard access where practical.

NFR-06 Maintainability: Algorithms, UI controls, and rendering logic should be separated into clear modules.

NFR-07 Security: The system should not execute user-provided code; input should be treated as data and validated.

5. External Interface Requirements

5.1 User Interface

The main screen should include an algorithm selector, input area, Run / Pause / Step / Reset controls, a visualization panel, pseudocode panel, and result/complexity area.

5.2 Software Interface

The UI calls the algorithm engine with validated input. The engine returns a sequence of state changes or step events. The visualization renderer consumes those events and updates the display.

5.3 Hardware Interface

No special hardware is required. A device capable of running a modern browser is sufficient.

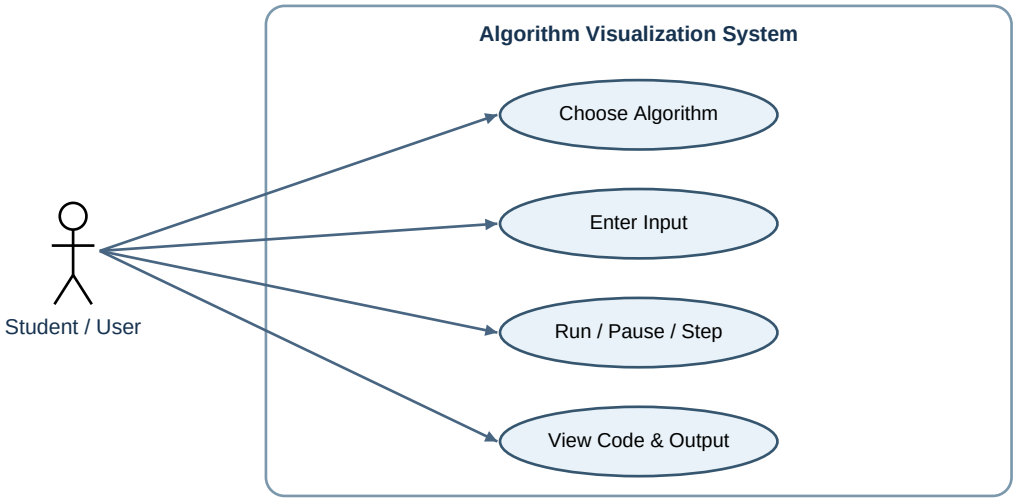
6. Main Use-Case Description

Use case: Visualize an algorithm
Primary actor: Student / user
Precondition: The application is open and an algorithm is available.
Main flow: (1) User selects an algorithm. (2) User enters input. (3) System validates input. (4) User starts visualization. (5) System displays each step and highlights relevant values. (6) System displays the result and complexity.
Alternative flow: If input is invalid, the system shows an error and asks the user to correct it. If the user pauses or resets, the visualization stops or returns to its initial state.
Postcondition: The user can review the execution and final result.

7. UML Diagrams

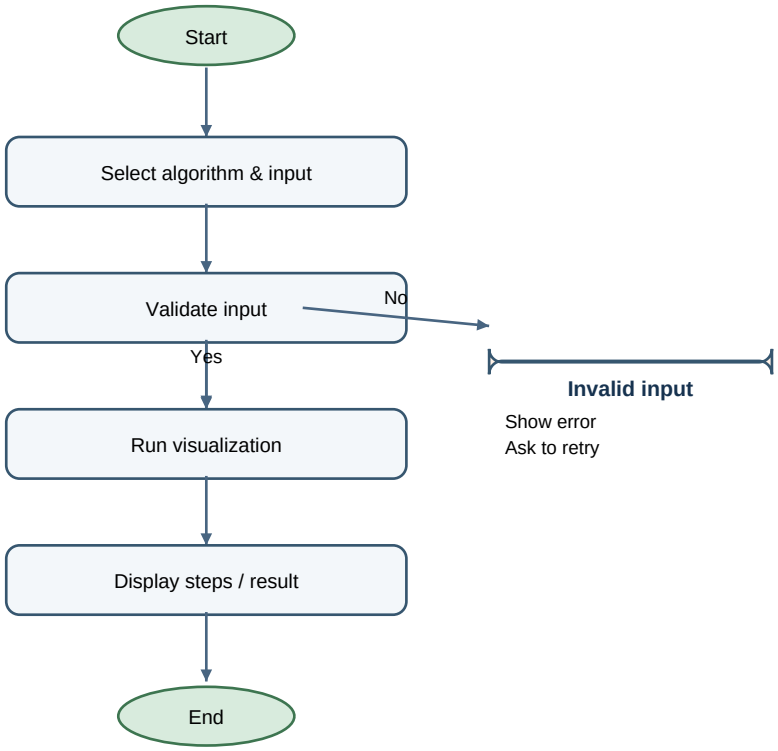
7.1 Use Case Diagram

Use Case Diagram



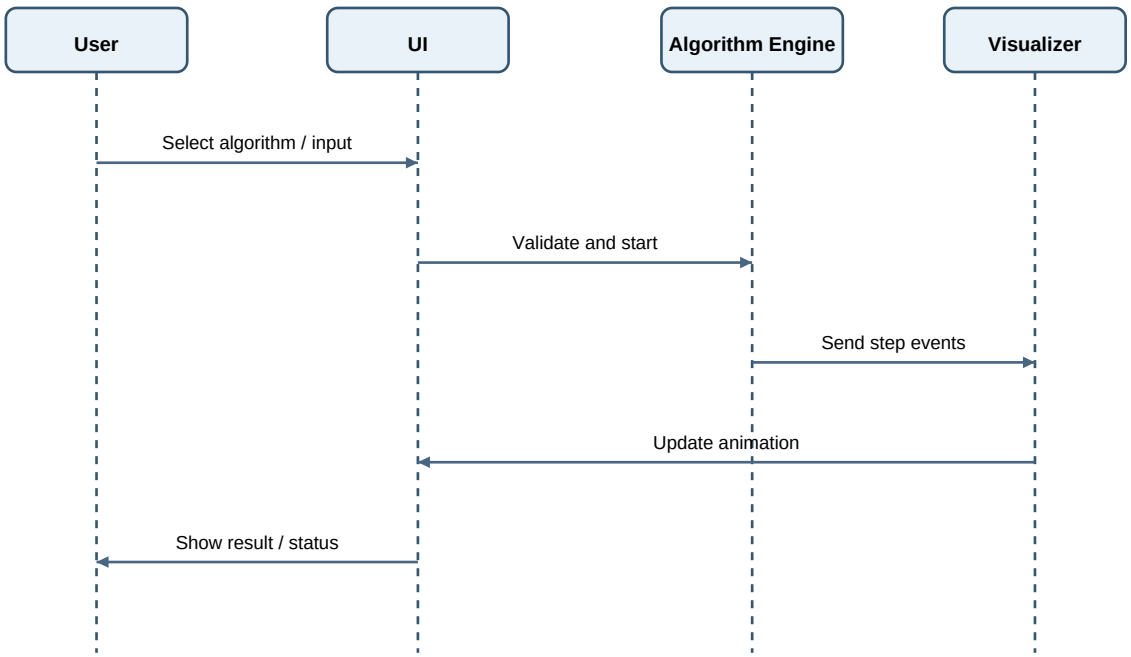
7.2 Activity Diagram

Activity Diagram



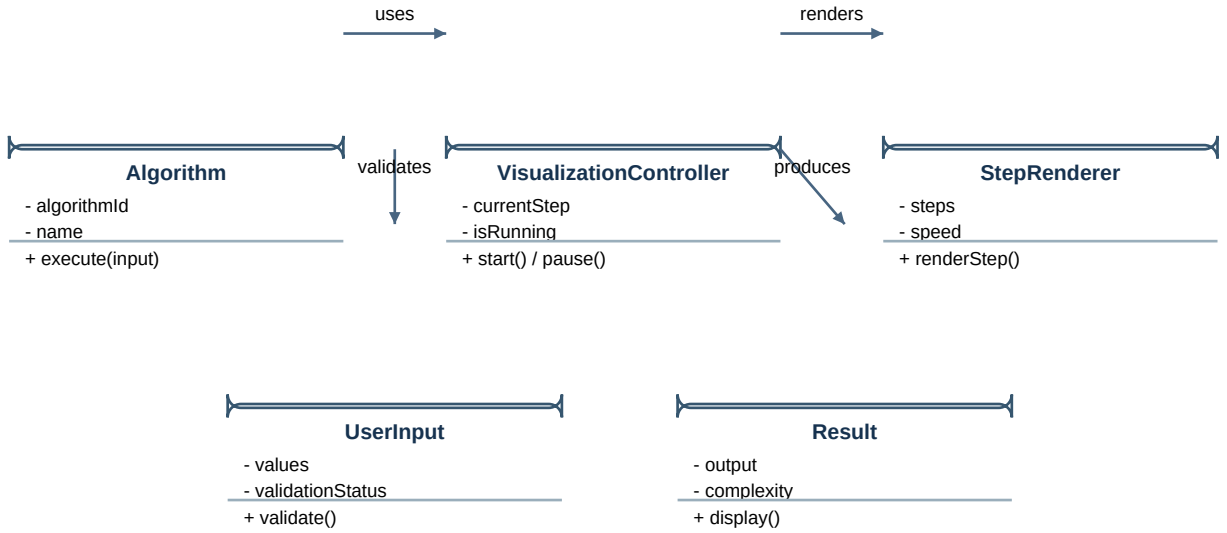
7.3 Sequence Diagram

Sequence Diagram



7.4 Class Diagram

Class Diagram



8. Data and Module Design

- **Algorithm Catalogue:** stores algorithm names, descriptions, and supported input types.
- **Input Validator:** checks input format and algorithm-specific constraints.
- **Algorithm Engine:** performs the algorithm and emits step events.
- **Visualization Controller:** controls play, pause, step, and reset behavior.
- **Step Renderer:** updates highlighted values, pointers, and panels.
- **Result Presenter:** displays output and complexity information.

9. Acceptance Criteria

1. A user can select at least one supported algorithm and provide valid input.
2. Invalid input produces a readable message instead of a crash.
3. Run, pause, step, and reset controls behave consistently.
4. The visual state matches the algorithm's current step.
5. The final result is correct for tested sample inputs.
6. The interface is usable on desktop and mobile screen widths.

10. Future Enhancements

Future versions may add more sorting and graph algorithms, speed control, side-by-side algorithm comparison, user accounts, saved runs, quizzes, and downloadable learning reports.

End of Document